Vapor Stone
RAIL SYSTEMS
A Wabtec company

10 655 Henri-Bourassa Blvd., West
Saint-Laurent (Québec) Canada
H4S 1A1

Tel: 514.335.4200
Fax: 514.335.4231

www.wabtec.com

Vapor Stone Rail Systems (VSRS)

Generic Software Documentation Project

Project # 7260

Portable Test Interface Protocol

Interface Control Document (ICD)

Contract Number		CDRL Number	
		Customer Ref. Number	

NOTICE
Plans, specifications and engineering data submitted herewith or which may be submitted, remain the property of **Vapor Stone Rail Systems** and shall not be reproduced or altered or used in whole or in part except by express consent of **VSRS**.

AVIS
Les plans, spécifications et données techniques soumis ci-joints, ou qui peuvent être soumis, restent la propriété de **Vapor Stone Rail Systems** et ne doivent pas être reproduits, altérés ou utilisés, totalement ou partiellement, sans le consentement express de **VSRS**.

	Name / Title	Signature / Date
Prepared :	James Kim/Thomas Marmeth Software Developers	2011-10-18 James Kim 201110-18
Verified :	Éric Malo, Eng. System Engineer	2011-10-18
Approved :	Martin Ouellet, Eng. Software Department Supervisor	2011-10-18

Document No.	**201111489**	Revision	**1**
		Date (yyyy-mm-dd)	**2011-10-17**

Template 200610247-00R-A

Revision List

Rev.	Date	By	ECD	Description of changes
1	2011-10-17	Thomas Marmeth		Released version.
0.1	2011-07-29	Éric Malo James Kim		Implementing comments from internal inspection I-1725. Updated for compliance to IEEE-1558-2004 (ICD section)
0	2011-06-23	James Kim	-	Original issue.

PREFACE

This document complies with the format and content requirements of the IEEE Std.1558-2004 standards for the ICD document.

Table of Contents

REVISION LIST **2**

PREFACE **2**

TABLE OF CONTENTS **3**

LIST OF FIGURES **5**

LIST OF TABLES **5**

1 INTRODUCTION **6**

1.1 PURPOSE 6
1.2 SCOPE 6
1.3 ABBREVIATIONS, ACRONYMS AND DEFINITION 6
1.3.1 Definitions *6*
1.3.1.1 Event Level Definitions 6
1.3.2 Acronyms *6*
1.4 REFERENCES 7

2 INTERFACES **8**

2.1 PORTABLE TEST APPLICATION COMMUNICATION INTERFACE 8
2.1.1 Interface Context *8*
2.1.2 Interface Requirements *8*
2.1.3 Interface Design *9*
2.1.3.1 Protocol Format 9
2.1.3.1.1 The CMD_XFER Field 9
2.1.3.1.2 The LENGTH Field 9
2.1.3.1.3 The SEQ_NO Field 9
2.1.3.1.4 The CTRL_ID Field 9
2.1.3.1.5 The COMMAND Field 10
2.1.3.1.6 The DATA Section 10
2.1.3.1.7 The CRC_16 Field 10
2.1.3.1.8 The EOM Field 10
2.1.3.2 Mandatory Commands 10
2.1.4 Protocol COMMANDS Description *12*
2.1.4.1 COMMAND Processing and Error Handling 12
2.1.4.2 Commands Description 14
2.1.4.2.1 Login and Logout Control Commands 14
2.1.4.2.1.1 PTA_LoginReq 14
2.1.4.2.1.2 PTA_LoginRsp 14
2.1.4.2.1.3 PTA_LogoutReq 14
2.1.4.2.1.4 PTA_LogoutRsp 15
2.1.4.2.1.5 PTA_ComTestReq 15
2.1.4.2.1.6 PTA_ComTestRsp 15
2.1.4.2.2 File Access Commands 16
2.1.4.2.2.1 PTA_FileReq 17
2.1.4.2.2.2 PTA_FileRsp 19
2.1.4.2.2.3 PTA_NextPackData 20
2.1.4.2.2.4 PTA_NextPackAck 20
2.1.4.2.2.5 PTA_LastPackData 20
2.1.4.2.2.6 PTA_FileXferOverAck 20

2.1.4.2.2.7 PTA_FileDeleteReq 21
2.1.4.2.2.8 PTA_FileDeleteRsp 21
2.1.4.2.3 PTA_EventSyncReq 21
2.1.4.2.4 PTA_EventSyncRsp 21
2.1.4.2.5 PTA_RtdmDataReq 22
2.1.4.2.6 PTA_RtdmDataRsp 22
2.1.4.2.7 PTA_VersionReq 23
2.1.4.2.8 PTA_VersionRsp 24
2.1.4.2.9 PTA_SciVersionRsp 24
2.1.4.2.10 PTA_LocationIDReq 25
2.1.4.2.11 PTA_LocationIDRsp 26
2.1.4.2.12 PTA_DMStoPTAErrorRsp 26
2.1.4.2.13 PTA_TimeReq 26
2.1.4.2.14 PTA_TimeRsp 26
2.1.4.2.15 PTA_SelfTestReq 27
2.1.4.2.16 PTA_SelfTestRsp 27
2.1.4.2.17 PTA_CodeStatusReq 27
2.1.4.2.18 PTA_CodeStatusRsp 28
2.1.4.2.19 PTA_DMSConfigReq 28
2.1.4.2.20 PTA_DMSConfigRsp 29
2.1.4.2.21 PTA_DoorCMDReq 29
2.1.4.2.22 PTA_DoorCMDRsp 29
2.1.4.2.23 PTA_SystemCountersResetReq 29
2.1.4.2.24 PTA_SystemCountersResetRsp 30

APPENDIX A: CRC CALCULATION ALGORITHM 31

List of Figures

Figure 2-1: Portable Test Application Typical Interface Overview 8
Figure 2-2: Error Handling 13
Figure 2-3: File Transfer Protocol Implementation – File Upload and No Error 16
Figure 2-4: File Transfer Protocol Implementation – File Download and No Error 17

List of Tables

Table 2-1: Data Exchange Format 9
Table 2-2: SEQ_NO field 9
Table 2-3: Mandatory Commands 10
Table 2-4: PTA_LoginRsp Data Definition 14
Table 2-5: PTA_LoginRsp status_code Field Values 14
Table 2-6: PTA_LogoutReq Data Codes 15
Table 2-7: PTA_LogoutRsp Data Byte Definition 15
Table 2-8: PTA_FileReq Data Bytes Definition 18
Table 2-9: Data Bytes Definition of the “Request” Field in PTA_file_req Structure 18
Table 2-10: Index for File Transfer 18
Table 2-11: PTA_FileRsp Data Bytes Definition 19
Table 2-12: Data Bytes Definition of the “Status” Field 19
Table 2-13: Info Returned when “FR_DIRECTORY_LOOKUP” Requested 20
Table 2-14: PTA_NextPackAck Command Format 20
Table 2-15: PTA_FileXferOverAck Data Byte 21
Table 2-16: PTA_EventSyncRsp Data Bytes Definition 21
Table 2-17: Event AttributeStruct Data Structure Definition 22
Table 2-18: Event ExtraAttribStruct Data Structure Definition 22
Table 2-19: PTA_RtdmDataRsp Response Data Structure 23
Table 2-20: PTA_VersionReq Data Bytes Definition 23
Table 2-21: PTA_VersionReq Field Name Definition 23
Table 2-22: PTA_VersionRsp Data Bytes Definition 24
Table 2-23: PTA_SciVersionRsp Data Bytes Definition 24
Table 2-24: PTA_SystemSciVersionStruct Data Bytes Definition 25
Table 2-25: Description and Values of Elements in PTA_SystemSciVersionStruct 25
Table 2-26: PTA_LocationIDRsp Data Bytes Definition 26
Table 2-27: PTA_DMStoPTAErrorRsp Data Bytes Definition 26
Table 2-28: PTA_DMStoPTAErrorRsp Error Codes 26
Table 2-29: PTA_SelfTestRsp Data Bytes Definition 27
Table 2-30: PTA_CodeStatusReq Data Byte Definition 27
Table 2-31: PTA_CodeStatusRsp Data Bytes Definition 28
Table 2-32: PTA_DMSConfigReq Data Bytes Definition 28
Table 2-33: PTA_DMSConfigReq *uCode* Field Values 28
Table 2-34: PTA_DMSConfigReq *uCommand* and *uiParameter* Fields Values 28
Table 2-35: PTA_DoorCMDReq Data Byte Definition 29
Table 2-36: PTA_DoorCMDRsp Data Byte Definition 29
Table 2-37: PTA_SystemCountersResetReq Data Byte Definition 30

1 INTRODUCTION

1.1 Purpose

This document describes the protocol used between the Portable Test Application (PTA) and any Door Module System (DMS) Application.

The intended audience is the Vapor Stone Rail Systems software engineering group and the maintenance team members.

1.2 Scope

This document is intended to be used as a reference for the development of the door system applications and the Portable Test Application.

This document specifies syntax and semantics of the communication protocol as well as procedures to use it.

1.3 Abbreviations, Acronyms and Definition

1.3.1 Definitions

Door Module System: A term that is used for generically identifying a processor based electronic module running any software within any DCU.

Door Control Unit: The final assembly/unit that is used to control one or more door panels. This unit is normally made of one or two door control modules running the Door Control Module Application and possibly one Communication Gateway Module Application.

Door Control Module: A specific processor based electronic module running a Door Control Module Application. It is used for controlling the door motor.

Communication Gateway Module: A specific processor based electronic module running a Communication Gateway Module Application. It acts as gateway between the Door Control Module Application and the Car Network/Portable Test Application.

1.3.1.1 Event Level Definitions

Event: Term used to globally identify an Info, an Exception or a Failure.

Info: Info is an occurrence of an informative condition monitored by the system, which is neither an exception nor a failure.

Exception: An exception is an occurrence of a condition that is outside of the specified parameters and affecting an outside Line Replaceable Unit (LRU).

Failure: A failure is an occurrence of a condition that is outside of the specified parameters and affecting this LRU. A failure normally requires maintenance or replacement to restore affected unit to its normal operation conditions.

1.3.2 Acronyms

Acronyms	Meaning
CGM	Communication Gateway Module
CPLD	Complex Programmable Logic Device

Acronyms	Meaning
CCITT	Comité Consultatif International Téléphonique et Télégraphique
CPU	Central Processing Unit
CRC	Cyclic Redundancy Check
CRC32	32-bit Cyclic Redundancy Check
DCM	Door Control Module
DCU	Door Control Unit
DMS	Door Module System
EDF	Event Definition File
EOM	End of Message
HW	Hardware
ICD	Interface Control Document
ID	Identification
LED	Light Emitting Diode
LRU	Line Replaceable Unit
LSB	Least Significant Bit
MSB	Most Significant Bit
OPDF	Operating Parameter Definition File
P/N	Part Number
PROM	Programmmable Read Only Memory
PTA	Portable Test Application
SACS	Software Application Configuration Specification
SCI	Software Configuration Item
SRS	Software Requirements Specifications
USB	Universal Serial Bus
VSRS	Vapor Stone Rail Systems

1.4 References

SACS Software Application Configuration Specification (Project specific)

2 INTERFACES

2.1 Portable Test Application Communication Interface

2.1.1 Interface Context

Figure 2-1 depicts the software system logical interfaces for the most of the projects in VSRS. In this figure, the grayed boxes are optional. The PTA may communicate with a Door Control Module (DCM) Application directly or through a Communication Gateway Module (CGM) Application. In addition, the Door Control Unit (DCU) may be implemented with one or two Door Control Modules (DCM) (one per door panel or one for both door panels) depending on the particular customer configuration.

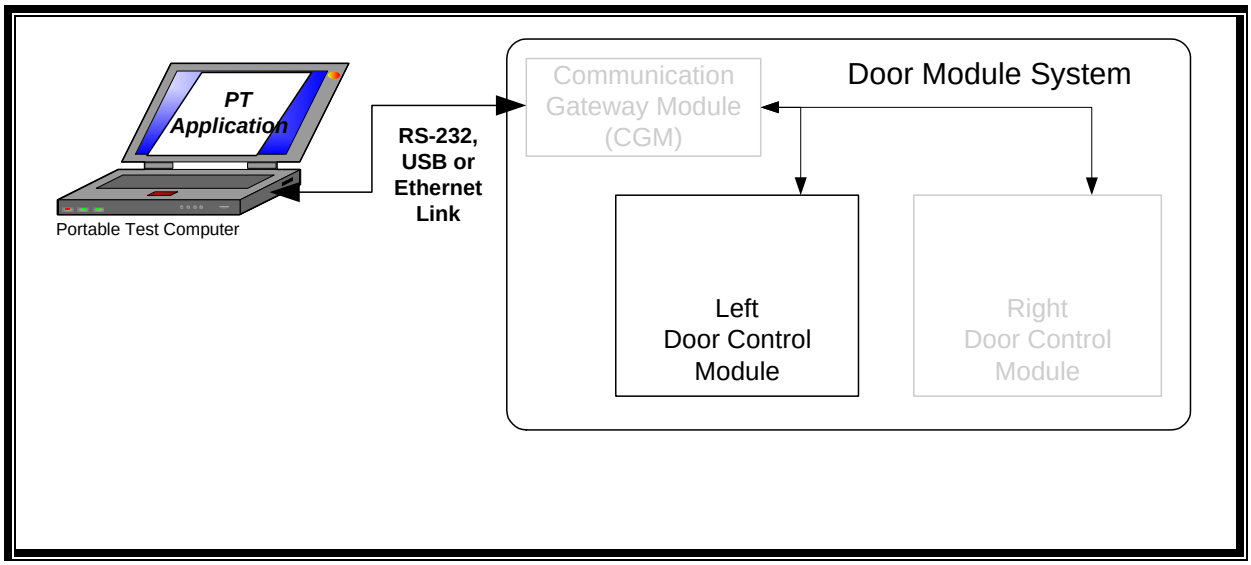


Figure 2-1: Portable Test Application Typical Interface Overview

2.1.2 Interface Requirements

The communication between the door module system and the PTA is implemented in a ***Master*** – ***Slave*** relationship. The Portable Test Application is always the ***Master***.

The protocol is based on ***Request*** / ***Response*** mode. The ***Slave*** end shall only send ***Response*** when requested by the PTA.

The following is a summary of the functions offered on this interface:

- Log downloaded from the door module
- Door module status data for Monitoring/Diagnostic
- Door module application configuration data
- Applications Upgrade (including Boot Loader)

2.1.3 Interface Design

2.1.3.1 Protocol Format

Following is a representation of the data exchange format:

Table 2-1: Data Exchange Format

0								1								2								3							
0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1
CMD_XFER = 0x01								LENGTH								SEQ_NO															
CTRL_ID								COMMAND								DATA 1								. . .							
DATA N								CRC_16																EOM = 0x00							

The door module system and Portable Test Application shall format the data for the data exchange in the Big Endian format.

The following sub-sections provide explanation on how to use each of these protocol fields.

2.1.3.1.1 The CMD_XFER Field

This protocol field represents the header of the message and shall always be transmitted as the first byte of the message. The value of CMD_XFER is 0x01.

2.1.3.1.2 The LENGTH Field

The LENGTH protocol field is the second byte of the message. It represents the number of data bytes in the message including SEQ_NO, CTRL_ID, COMMAND, variable length of DATA section and CRC_16 fields. It does not include CMD_XFER, LENGTH (itself) and EOM bytes.

2.1.3.1.3 The SEQ_NO Field

This field is used to distinguish messages from previously transmitted messages as in a sliding window protocol. The SEQ_NO field is a two bytes field using the following format:

Table 2-2: SEQ_NO field

```
- PTA sends Packet:
-       PTA_Seq range 1~255, increased by one.
-       DMS_Seq is set to 0.
- DMS sends Packet:
-       When the packet is response to specific request of PTA, PTA_Seq should be same as PTA packet
PTA_Seq.
-       DMS_Seq is DMS packet number. Range 0~255, increased by one.

typedef struct {
        char    PTA_Seq;        // 1 Byte
        char    DMS_Seq;        // 1 Byte
} Seq_No;
```

2.1.3.1.4 The CTRL_ID Field

The CTRL_ID protocol field is a one-byte binary number. When more than one Door Control Module (DCM) is available, this field is used in the Request from the Master, to specify to which DCM the message is directed. The same field is used in the Response from the Slave, to specify from which DCM the message is coming from.

When only one DCM is available, this protocol field shall always be set to 0. When more than 1 DCM is available, this field will be defined in the *Software Application Configuration Specification* (SACS) document.

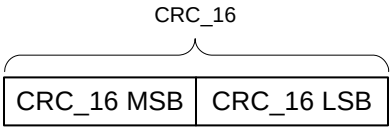
2.1.3.1.5 The COMMAND Field

The Command section is the first significant byte of the message. It describes what signification has the data section. This field is used to issue a specific command. (See Table 2-3 Section 2.1.3.2)

2.1.3.1.6 The DATA Section

The data section represents actual data bytes. When no data is to be passed, the data section is empty.

2.1.3.1.7 The CRC_16 Field



The CRC_16 field is used to ensure the integrity of the message. For CRC_16 calculation, CMD_XFER and EOM shall not be part of the calculation; therefore, the CRC calculation is performed on the following fields:

- LENGTH,
- SEQ_NO,
- CTRL_ID,
- COMMAND,
- DATA 1 to N (when available)

The type of used CRC is 16 bits CRC according to the CRC16 CCITT divisor polynomial. To avoid compatibility issues, application should use algorithm and CRC table provided in Appendix A.

2.1.3.1.8 The EOM Field

This protocol field represents the trailer of the message and shall always be transmitted as the last byte of the message. The value of EOM is 0x00.

2.1.3.2 Mandatory Commands

The commands in Table 2-3 below are mandatory commands that should be supported by all applications. Please note that these mandatory commands are not applying to boot loaders that may support only a sub-set of these commands. In this table, the HW_COM Mnemonic refers to the function identification used in the SRS documents.

If it is required, optional protocol commands are listed in the SACS document. The optional protocol may be implemented on a project's basis depending on project's requirements.

For the additional project specific commands, the command code shall be in the range from 200 to 254.

Table 2-3: Mandatory Commands

HW_COM Mnemonic	COMMAND Code (Decimal)	COMMAND Name
LoginRequest	70	PTA_LoginReq
LoginResponse	71	PTA_LoginRsp
LogoutRequest	72	PTA_LogoutReq
LogoutResponse	73	PTA_LogoutRsp
RebootRequest	72	PTA_LogoutReq
LogoutResponse	73	PTA_LogoutRsp
LocationIdentificationRequest	48	PTA_LocationIDReq

HW_COM Mnemonic	COMMAND Code (Decimal)	COMMAND Name
LocationIdentificationResponse	49	PTA_LocationIDRsp
CarNumberRequest	48	PTA_LocationIDReq
CarNumberResponse	49	PTA_LocationIDRsp
LocalTimeRequest	64	PTA_TimeReq
LocalTimeResponse	65	PTA_TimeRsp
SelfTestRequest	66	PTA_SelfTestReq
SelfTestResponse	67	PTA_SelfTestRsp
CodeStatusRequest	74	PTA_DMSCodeStatusReq
CodeStatusResponse	75	PTA_DMSCodeStatusRsp
HeartBeatRequest	79	PTA_ComTestReq
HeartBeatResponse	80	PTA_ComTestRsp
MonitoringRequest	13	PTA_DataReq
MonitoringResponse	14	PTA_DataRsp
EventStatusRequest	7	PTA_EventSyncReq
EventStatusResponse	8	PTA_EventSyncRsp
EraseEventLogFileRequest EraseSnapshotFileRequest DeleteCommunicationGateway-LogRequest	69	PTA_FileDeleteReq
EraseEventLogFileResponse EraseSnapshotFileResponse DeleteCommunicationGateway-LogResponse	78	PTA_FileDeleteRsp
EventLogFileRequest SnapshotFileRequest EventDescriptionFileRequest OperatingParameters-DescriptionFileRequest DoorMovementInformation-FilesRequest SystemCounterFileRequest MemoryDumpRequest	3 32 34	PTA_FileReq PTA_NextPackData PTA_LastPackData
EventLogFileResponse SnapshotFileResponse EventDescriptionFileResponse OperatingParameters-DescriptionFileResponse DoorMovementInformation-FilesResponse SystemCounterFileResponse MemoryDumpResponse	4 33 35	PTA_FileRsp PTA_NextPackAck PTA_FileXferOverAck
SoftwareVersionsRequest	16	PTA_VersionReq
SoftwareVersionsResponse	18	PTA_VersionRsp
ReadDMSConfiguration-Request WriteDMSConfiguration-Request	76	PTA_DMSConfigReq
ReadDMSConfiguration-Response WriteDMSConfiguration-Response	77	PTA_DMSConfigRsp
DoorCyclesCounterReset-Request OperatingHoursReset-Request FaultOccurrenceCounters-ResetRequest	83	PTA_SystemCountersResetReq
DoorCyclesCounterReset-Response OperatingHoursReset-Response FaultOccurrenceCounters-ResetResponse	84	PTA_SystemCountersResetRsp
DoorCommandRequest	81	PTA_DoorCMDReq
DoorCommandResponse	82	PTA_DoorCMDStatus
DoorCommandErrorResponse	50	PTA_DMStoPTAErrorRsp

2.1.4 Protocol COMMANDS Description

2.1.4.1 COMMAND Processing and Error Handling

This protocol does not use acknowledgement. No time out is associated to the transmission of a message on the door module side. If a CRC error or an invalid data is encountered, the ***Slave*** end shall ignore the command. If the bad command is received, the ***Slave*** end shall send PTA_DMStoPTAErrorRsp. When a COMMAND is not correctly responded, the requesting end may implement a maximum of two retries within one second.

The sequence diagram presented in Figure 2-2 on the next page is providing a graphical representation of this concept.

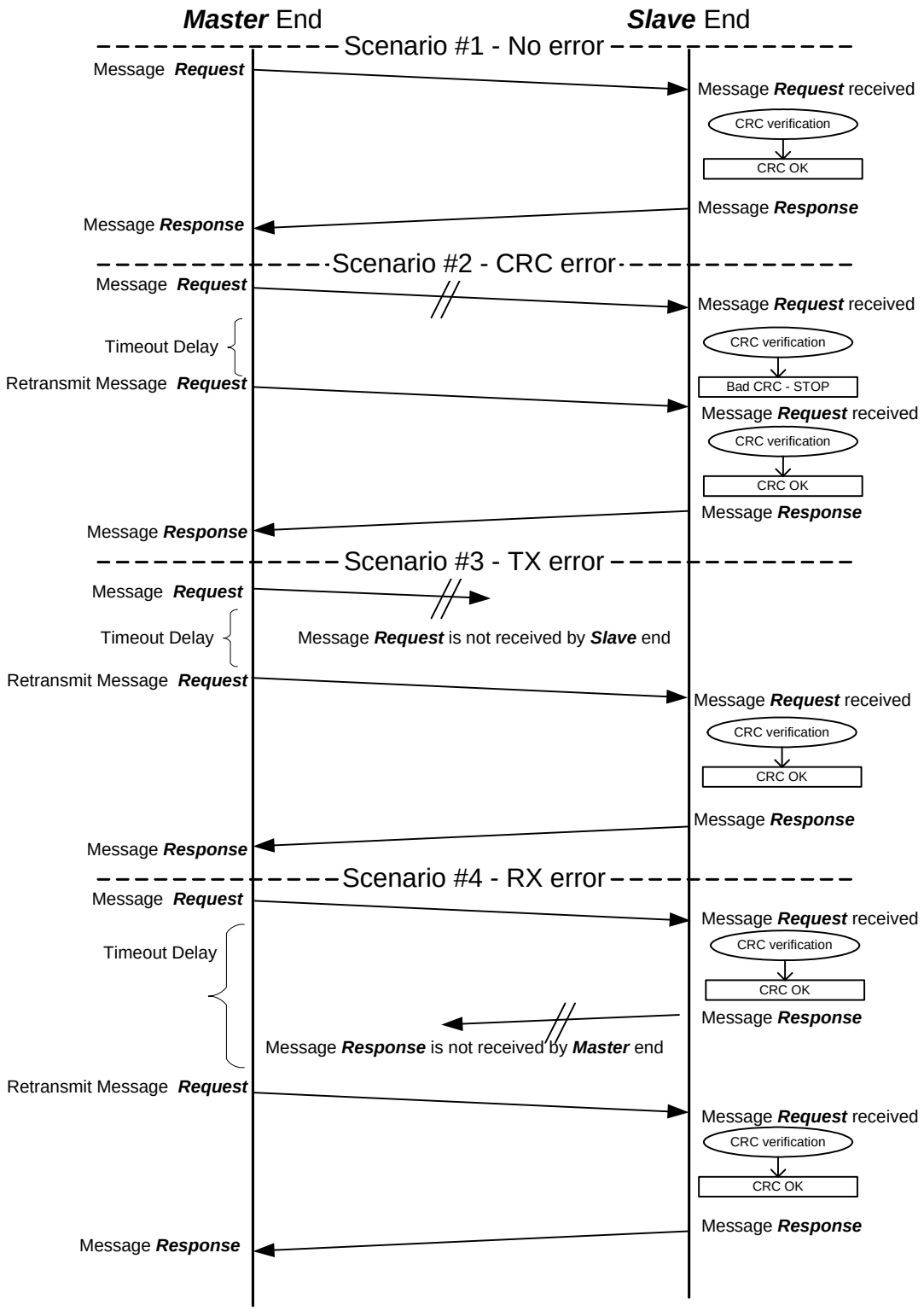


Figure 2-2: Error Handling

2.1.4.2 Commands Description

2.1.4.2.1 Login and Logout Control Commands

2.1.4.2.1.1 PTA_LoginReq

COMMAND Value: 70

DATA LENGTH: 80 bytes

Master end shall send this command request to the ***Slave*** end to login and have access to others commands. This must be the first command sent to the ***Slave*** end.

The data section of this command represents the login string to be used for accessing system's functions and shall have 80 bytes, padded with zeros if shorter than 36 bytes.

The login string is project specific and is outside the scope of this protocol.

2.1.4.2.1.2 PTA_LoginRsp

COMMAND Value: 71

DATA LENGTH: 80 bytes

The ***Slave*** end shall send this command response to the Master end in response to the PTA_LoginReq command request. The data section of this command response contains two fields as defined in Table 2-4. The status_code field shall use one of the values from Table 2-5. The sci_description field shall be the same as the login string sent from the PTA_LoginReq command request to be validated.

Table 2-4: PTA_LoginRsp Data Definition

```
typedef struct {

             unsigned       status_code; // 1 Byte
             // SCIDESCRIPTIONLENGHT is 45
             char           sci_description[SCIDESCRIPTIONLENGHT];

} Login_Status;
```

Table 2-5: PTA_LoginRsp status_code Field Values

Value	Description
0	Reserved
1	Error occurred
2	Login Boot Loader OK
3	Login Application OK
4	Login Boot loader and Application is not presented

2.1.4.2.1.3 PTA_LogoutReq

COMMAND Value: 72

DATA LENGTH: 1 byte

The ***Master*** end shall send this command request to the ***Slave*** end to logout from the current session.

The data section of this command is 1byte long and can take one of the two values specified in Table 2-6 below.

Table 2-6: PTA_LogoutReq Data Codes

Value	Description
0	Normal logout request
1	Logout and reboot

2.1.4.2.1.4 PTA_LogoutRsp

COMMAND Value: 73

DATA LENGTH: 1 byte

The ***Slave*** end shall send this command response to the ***Master*** end in response to the PTA_LogoutReq command request. The ***Slave*** end shall also send this command when the ***Master*** end sends a valid command without being logged. In that case, the ***Slave*** end shall send the status code 0 as defined in Table 2-7 below. Other specific status codes are define in the Table 2-7 below.

The ***Slave*** end may implement automatic logout when no commands are received for a specific time out. These particular behaviors are outside the scope of this protocol and may be implemented as long as the implementation is compliant with this protocol document. See also next section 2.1.4.2.1.5 below.

Table 2-7: PTA_LogoutRsp Data Byte Definition

Value	Description
0	Portable Test was not logged in
1	Error occurred
2	Logout Boot Loader OK
3	Logout Application OK
4	Door module reboot will be in progress

2.1.4.2.1.5 PTA_ComTestReq

COMMAND Value: 79

DATA LENGTH: 0 byte

In the case where an automatic time out is implemented, this command request (which is doing nothing else than transmitting an empty request) can be used by the ***Master*** end to keep the connection alive. It can be considered as a “HeartBeat” signal.

2.1.4.2.1.6 PTA_ComTestRsp

COMMAND Value: 80

DATA LENGTH: 0 byte

The ***Slave*** end shall send this command response to the ***Master*** end in response to the PTA_ComTestReq command request.

2.1.4.2.2 File Access Commands

File access uses the commands described in the next subsections. Supported types of files are as follows:

- Application files,
- Memory,
- Event Description File (EDF),
- Door Control Module event log file,
- Operating Parameters Definition File (OPDF),
- Communication Gateway Module event log file,
- Snapshot files (Real-Time data associated to events in the logs),
- Door movement information file (Graphical data of door position/speed/current vs time),
- System counters

The following explains how the File Transfer Protocol is implemented. The diagram shows a transmission without error:

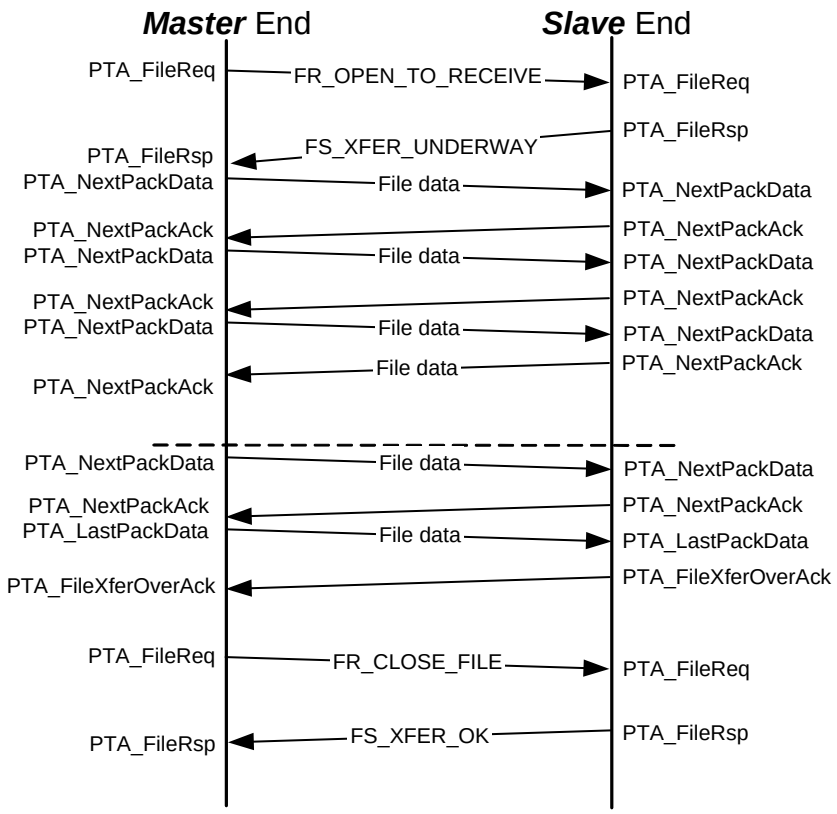


Figure 2-3: File Transfer Protocol Implementation – File Upload and No Error

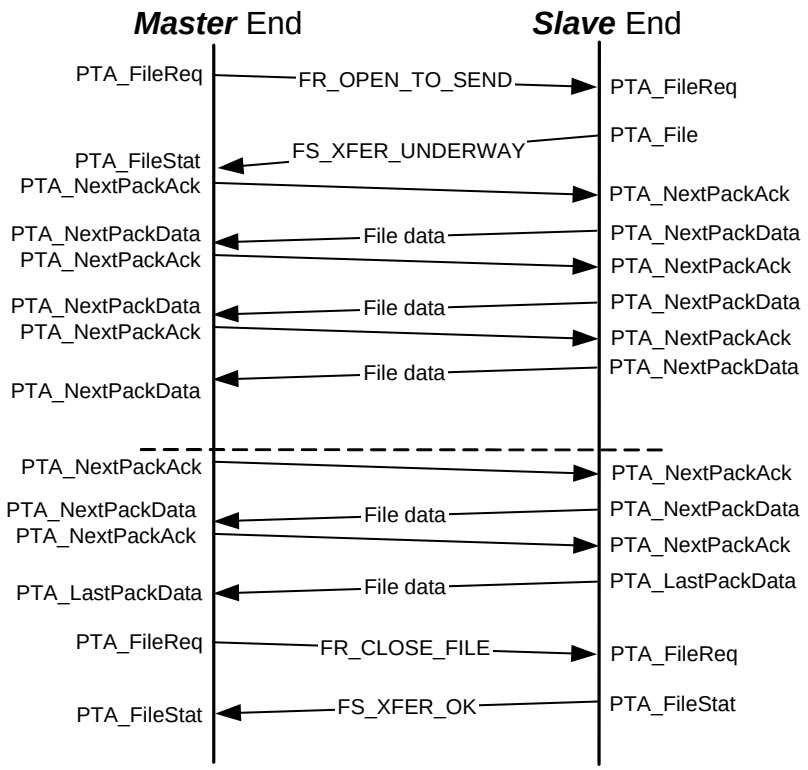


Figure 2-4: File Transfer Protocol Implementation – File Download and No Error

2.1.4.2.2.1 PTA_FileReq

COMMAND Value: 03

DATA LENGTH: 12 bytes

The ***Master*** end shall send this command to the ***Slave*** end to initiate a file transfer or request a directory lookup.

The data section of this command describes the operation to be performed on a given file and is defined in Table 2-8, Table 2-9 and Table 2-10. Table 2-8 shows the data structure used as the DATA section. Table 2-9 shows the valid ranges for the ***request*** field of the DATA section which is specifying the file operation to be performed. The Table 2-10 provides valid values for the ***index*** field which is used to identify the type of file on which the request operation is to be performed.

The 9 bytes ***ramDump*** field as seen in Table 2-8 is valid only with ***index*** value = 1 for specifying the memory location and memory size to be retrieved.

Please note that this protocol supports only one file transfer at a time. If a file transfer is requested while another one is already in progress, the sender and the receiver shall ignore the request. In the particular case where the ***Slave*** end receives a FR_OPEN_TO_SEND while a file transfer is already in progress, the ***Slave*** end shall respond with the status FS_ACCESS_UNAVAIL to the ***Master*** end.

Table 2-8: PTA_FileReq Data Bytes Definition

```
typedef struct {
      char            request;              // 1 Byte
      unsigned        int index;            // 2 Bytes
      union
      {
              Unsigned char reserved[9];    // 9 Bytes
              Struct
              {
                      Unsigned long         ulAddress;
                      Unsigned long         ulLength;
                      Unsigned char         ucUnused;
              }ramDump;
      }reserved;
} PTA_file_req;
```

Table 2-9: Data Bytes Definition of the "Request" Field in PTA_file_req Structure

Value	Identifier	Notes
0	FR_OPEN_TO_SEND	Open the indicated file for sequential reading
1	FR_OPEN_TO_RECEIVE	Open the indicated file for writing. Only for Boot Loader
2	FR_CLOSE_FILE	Close and save file
4	FR_DIRECTORY_LOOKUP	Retrieve directory entry
0xFF	FR_NUL	Value not available

In the case of a "FR_OPEN_TO_RECEIVE" file request, the ***Master*** end shall be ready to send the requested file immediately after the reception of the PTA_FileResp with the FS_XFER_UNDERWAY status from the ***Slave*** end. Data is then exchanged by alternately using the PTA_NextPackData and PTA_NextPackAck commands. When the transfer is fully completed, the ***Slave*** end shall inform the ***Master*** end of the transfer completion by issuing a PTA_FileXferOverAck command. Please refer to Figure 2-3 for the complete sequence diagram of a file upload.

In the case of a "FR_OPEN_TO_SEND" file request, the ***Slave*** end shall be ready to send the requested file at the next PTA_NextPackAck command from the ***Master*** end. Data is then exchanged by alternately using the PTA_NextPackData and PTA_NextPackAck commands. Please refer to Figure 2-4 for the complete sequence diagram of a file download.

Table 2-10: Index for File Transfer

File Index	File type	Description
0	2	Application code transferred to the Boot loader
1	1	Memory dump
2	257	Event Description File (EDF)
3	258	Internal Event Log File
4	259	Operating Parameters Definition File (OPDF)
5	260	Network Log File
6 -> 25	261	Snapshot file #1 to 20
26	262	Door Movement Information
27	262	Door Movement Information
28	263	System Counters
29 -> 63	-	Reserved
64	500	DCM configuration file
65	501	CGM configuration file

2.1.4.2.2.2 PTA_FileRsp

COMMAND Value: 04

DATA LENGTH: 28 bytes

The ***Slave*** end shall send this command response to the ***Master*** end to answer a PTA_FileReq command.

All file transfer commands shall be ended with the transmission of a command response with the 'request' data field sets to FILE_CLOSE_FILE value as illustrated in Figure 2-3 and Figure 2-4.

The data section of this command is described in the Table 2-11and Table 2-12 below.

The Data byte flash_sector is only used when the flash memory is erasing. The value presents the different flash sector.

Table 2-11: PTA_FileRsp Data Bytes Definition

```
typedef struct {
      char                status;           // 1 Byte
      char                flash_sector      // 1 Byte
      unsigned  int       number_of_files;  // 2 Bytes
      unsigned  int       selected_file;    // 2 Bytes
      struct {
                char      file_info[16];    // 16 Bytes
                unsigned  long       size;  // 4 Bytes
                unsigned  int        type;  // 2 Bytes
      } DirLookup;                          // 22 Bytes total for DirLookup structure
} PTA_file_status;
```

Table 2-12: Data Bytes Definition of the "Status" Field

Value	Identifier	Notes
0	FS_XFER_OK	Used as the command response value in response to the PTA_FileReq message request with FR_CLOSE_FILE at the end of a successful file transfer.
1	FS_LOOKUP_OK	Used as the command response value in response to the PTA_FileReq message request with FR_DIRECTORY_LOOKUP after a successful directory lookup.
3	FS_LOOKUP_ERR	Used as the command response value in response to the PTA_FileReq message request with FR_DIRECTORY_LOOKUP after a NON-successful directory lookup.
4	FS_XFER_UNDERWAY	Used as the command response value in response to the PTA_FileReq message request with FR_OPEN_TO_SEND or FR_OPEN_TO_RECEIVE.
5	FS_IO_ERR	Used as the command response value in response to the PTA_FileReq message request whenever on error is encountered while reading or writing the file.
6	FS_TIMEOUT_ERR	Used as the command response value in response to the PTA_FileReq message request whenever a time out is reached during a file transfer.
9	FS_ACCESS_UNAVAIL	Used as the command response value in response to the PTA_FileReq message request with FR_OPEN_TO_SEND of FR_OPEN_TO_RECEIVE when a file transfer is already in progress.
15	FS_ERASE_FLASH	Erase the flash memory to receive the file
0xFF	FS_NUL	Value not available / UNUSED

File transfer index number shall start at **2** and shall be incremented by 1 each time an additional file is requested for the door module functions. Each file is consecutively indexed.

Table 2-13: Info Returned when "FR_DIRECTORY_LOOKUP" Requested

Field name	Range	Definition
file_info	16 bytes char	Represents the file name. For a snapshot file, the file name corresponds to the time of the event that triggered the snapshot. This time is expressed in standard "C" type. This is a 32 bit number that represents the number of seconds elapsed since the January 1st, 1970. Note that this structure is copied in this field starting at field at file_info[0].
size	Unsigned long	Represents the size of the file in bytes.
type	Unsigned int	Represents the type of the file. See column #2 of Table 2-10.

2.1.4.2.2.3 PTA_NextPackData

COMMAND Value: 33

DATA LENGTH: 192 bytes

This 'data' command is used to transfer data in BOTH directions. This command performs the transfer of a block of 192 bytes of data.

The data section of this command is never empty.

2.1.4.2.2.4 PTA_NextPackAck

COMMAND Value: 32

DATA LENGTH: 0 bytes

Table 2-14: PTA_NextPackAck Command Format

CMD_XFER	LENGTH	SEQ. NO.	Controller ID	COMMAND #32	CRC 16	EOM

This 'acknowledge' command is used to confirm the reception of a data packet in response to the PTA_NextPackData. This command acknowledge the transfer of a block of 192 bytes of data.

2.1.4.2.2.5 PTA_LastPackData

COMMAND Value: 34

DATA LENGTH: Variable, up to 192 bytes

This 'data' command is used to transfer data in BOTH directions. This command performs the transfer of the last data block of the file.

The data section of this command is 0 to 192 bytes.

2.1.4.2.2.6 PTA_FileXferOverAck

COMMAND Value: 35

DATA LENGTH: 0 bytes

This 'acknowledge' response is sent exclusively from the ***Slave*** end to the ***Master*** end to inform the ***Master*** end that the file transfer operation is completed, successfully or not.

The data section of this command is one byte long. The data byte values are as showed in Table 2-15 below :

Table 2-15: PTA_FileXferOverAck Data Byte

Value	Description
0	Reserved.
1	No error during file transfer.
2	Error during transfer. Transfer operation aborted.

2.1.4.2.2.7 PTA_FileDeleteReq

COMMAND Value: 69

DATA LENGTH: 1 byte

The ***Master*** end shall send this command request to the ***Slave*** end to delete the Event log file, the Network Log file or any Snapshot file present.

Definition and range of the data byte shall be as per Table 2-10.

2.1.4.2.2.8 PTA_FileDeleteRsp

COMMAND Value: 78

DATA LENGTH: 1 byte

The ***Slave*** end shall send this command response to the ***Master*** end to acknowledge the deletion of the file requested in the PTA_FileDeleteReq command request.

2.1.4.2.3 PTA_EventSyncReq

COMMAND Value: 07

DATA LENGTH: 1 byte

The ***Master*** end shall send this command request to the ***Slave*** end to request a re-synchronization of the events that are in fault state. This command request is implemented as a “GetFirst/GetNext” type of command. Data byte value 1 representing the “GetFirst” while data byte value 0 representing the “GetNext”.

2.1.4.2.4 PTA_EventSyncRsp

COMMAND Value: 08

DATA LENGTH: 0 or 11 bytes

The ***Slave*** end shall send this command response to the ***Master*** end to transmit the first or next event in response to the PTA_Event SyncReq command request. Whenever the data length is less than 11 bytes, the ***Master*** end shall consider that it is the last entry.

Table 2-16: PTA_EventSyncRsp Data Bytes Definition

```
-   The “ByteType “  and “UcharType” types represent an 8-bit size.
-   The “UInt16Type” type represent an unsigned integer of 16-bit size.

#define LOCATION_LEN   6                          // Length of the location string.

typedef struct {
    UInt16Type          aEventCode;               // The event code. (2 bytes)
    AttributeStruct     aAttributes;              // The attributes of the event. (2 bytes)
    ExtraAttribStruct   aExtraAttribute;          // Extra attributes of the event. (1 byte)
    UCharType           aLocation[LOCATION_LEN];  // Node’s LOCATION_ID string. (6 bytes)
} SystemEventStruct;
```

Table 2-17: Event AttributeStruct Data Structure Definition

```
-   The “ByteType “ type represent an 8-bit size.


typedef struct {
    ByteType    Reserved1;//Reserved for future use
    ByteType    Reserved2;//Reserved for future use

} AttributeStruct;
```

Table 2-18: Event ExtraAttribStruct Data Structure Definition

```
-   The “ByteType “ type represent an 8-bit size.

typedef struct {
    ByteType      aType       :1;            // This bit field indicates the type
                                             // of event: 1 = Fault / Reset.
    ByteType      aState      :1;            // Indicates the state of the event:
                                             // (This field is irrelevant if
                                             // the event type is “Pulse”).
                                             // 0 = Reset.
                                             // 1 = Fault.
    ByteType      aSyncReq    :1;            // This bit field indicates if the
                                             // event has been issued following a
                                             // request:
                                             // 0 = Event issued normally.
                                             // 1 = Event issued following a
                                             // request.
    ByteType      aSpare      :1;            // This field is spare
    ByteType      aFile       :4;            // This field is used to indicate the
                                             // index of the snapshot file related
                                             // to the event. A zero ( 0 )
                                             // indicates no file is related to
                                             // this event.
} ExtraAttribStruct;
```

2.1.4.2.5 PTA_RtdmDataReq

COMMAND Value: 13

DATA LENGTH: 0 byte

The ***Master*** end shall send this command request to the ***Slave*** end to request the current values of the operating parameters. This command request is used to get real-time states of inputs and outputs from the ***Slave*** end.

2.1.4.2.6 PTA_RtdmDataRsp

COMMAND Value: 14

DATA LENGTH: 10 bytes

The ***Slave*** end shall send this command response to the ***Master*** end in response to the PTA_RtdmDataReq command request.

The data is an 80 bits bit field (10 bytes), as showed in Table 2-19 below, representing the state of inputs/outputs as defined by the Operating Parameters Description File (OPDF). See also section 2.1.4.2.2.1 and Table 2-10 for details about the OPDF.

Table 2-19: PTA_RtdmDataRsp Response Data Structure

```
typedef struct _Ldts_RecordStruct
{
            unsigned int uiSignal0;
            unsigned int uiSignal1;
            unsigned int uiSignal2;
            unsigned int uiSignal3;
            unsigned int uiSignal4;

} Ldts_RecordStruct;
```

2.1.4.2.7 PTA_VersionReq

COMMAND Value: 15

DATA LENGTH: 2 bytes

The Master end shall send this command request to the Slave end to request the version information of an installed Software Configuration Item (SCI). The data section shall be formatted as per Table 2-20 below where each field is defined in Table 2-21.

Table 2-20: PTA_VersionReq Data Bytes Definition

```
- The “ByteType “ type represent an 8-bit size.

typedef struct {
       ByteType                              aVersionRequest;
       ByteType                              aSciIndex;

} PTA_VersionReqStruct;
```

Table 2-21: PTA_VersionReq Field Name Definition

Field Name	Value	Definition
aVersionRequest	1. VERSION_REQUEST = Request for the EDF and OPDF versions. 2. SCI_VERSION_REQUEST = Request for the SCI versions. 3. SCI_LAST_VERSION_REQUEST = Request to send only the last SCI index of the DMS.	This field represents which version to send. In the case of a value of VERSION_REQUEST, only the EDF and OPDF version are sent. In the case of a value of SCI_VERSION_REQUEST, the “aSciIndex” field is used as an identifier of the SCI requested. The SCI_LAST_VERSION_ REQUEST is a request to send the last SCI version index of the DMS. This allows the PTA to know the maximum number of SCI’s contained in the DMS.

Field Name	Value	Definition
aSciIndex	Request for all available SCI Version Records. The records shall be issued starting from the highest index down to the lowest one (1) using "aSciVersionNumber" as the record index. a. 1-30: Specific SCI Version Record Index. Only send the SCI Version Record corresponding to the requested Index. b. 31-255: Not Used	This field represents the SCI index requested by the ***Master*** end. This field is used only if the preceding field "aVersionRequest" is set to SCI_VERSION_REQUEST. Otherwise, it is ignored.

2.1.4.2.8 PTA_VersionRsp

COMMAND Value: 16

DATA LENGTH: 4 bytes

The Slave end shall send this command request to the Master end in response to the PTA_VersionReq command request in the case where the "aVersionRequest" field (see Table 2-21) is equal to VERSION_REQUEST. The data format is defined in Table 2-22 below.

Table 2-22: PTA_VersionRsp Data Bytes Definition

```
- The "ByteType " type represent an 8-bit size.
typedef struct
{
        ByteType aMajorEdfVersion;     // Major version of the EDF file.
        ByteType aMinorEdfVersion;     // Minor version of the EDF file.

        ByteType aMajorOpParamVersion; // Major version of the operational parameter file.
        ByteType aMinorOpParamVersion; // Minor version of the operational parameter file.
} PTA_SystemVersionStruct;
```

2.1.4.2.9 PTA_SciVersionRsp

COMMAND Value: 17

DATA LENGTH: 206 bytes

The ***Slave*** end shall send this command response to the ***Master*** end in response to the PTA_VersionReq command request, in the case where the "aVersionRequest" field (see Table 2-21) is equal to SCI_VERSION_REQUEST.

The data section contains a collection of a maximum of 10 SCI versions. The data format is defined in Table 2-23, Table 2-24 and Table 2-25 below.

Table 2-23: PTA_SciVersionRsp Data Bytes Definition

```
typedef struct _SciVersionAns
{
    Seq_No          Sequence;                // Sequence structure, 2 bytes
    unsigned char   ucDoorID;                // Left (0) or Right (1)
    unsigned char   ucCommand;               // Command.
    PTA_SystemSciVersionStruct  Ans[10];     // Data byte of the SCI's.

}SciVersionAns;
```

Table 2-24: PTA_SystemSciVersionStruct Data Bytes Definition

```
- The “ByteType “ and “UCharType” type represent an 8-bit size.
- The “UInt16Type” type represents a 16-bit size.

typedef struct _PTA_SystemSciVersionStruct
{
        ByteType       aSciIndex;
        ByteType       aType;
        UCharType      aAcronym[8];
        UInt16Type     aYear;
        ByteType       aMonth;
        ByteType       aDay;
        UCharType      aRevision[6];
} PTA_SystemSciVersionStruct;
```

Table 2-25: Description and Values of Elements in PTA_SystemSciVersionStruct

Field Name	Value	Definition
AsciIndex	0 Not Used 1 – 30 SCI Index 31 – 255 Not Used.	This field represents the index of the SCI and is used to identify the SCI defined by this data element.
Atype	0 Firmware version 1 Software version 2 Externally programmable CPLD 3 Other 4 – 255 Not Used.	This field defines the type of the SCI: Firmware Version: software which requires a component replacement in order to be updated (ex. boot PROM, low-level driver, etc.) Software Version: software which can be updated directly on the equipment using a PTA (typically the application software).
aAcronym[8]	Alphanumeric	Acronym of the SCI. String shall be terminated by a Null character if shorter than eight (8) characters.
aYear / aMonth / aDay	aYear:16-bit value; aMonth: 1 – 12; aDay: 1 – 31;	The date of the revision.
aRevision[6]	Alphanumeric	This is a string representing the version number of the SCI. String shall be terminated by a Null character if shorter than six (6) characters.
aReserved1 / aReserved2	0	These fields are not used.

2.1.4.2.10 PTA_LocationIDReq

COMMAND Value: 48

DATA LENGTH: 0 byte

The ***Master*** end shall send this command request to the ***Slave*** end to request the Location_ID information. (As an example the Car ID together with the Door ID).

2.1.4.2.11 PTA_LocationIDRsp

COMMAND Value: 49

DATA LENGTH: 3 bytes

The ***Slave*** end shall send this command response to the ***Master*** end in response to the PTA_Location_IDReq command request. The data format is defined in Table 2-26 below.

Table 2-26: PTA_LocationIDRsp Data Bytes Definition

```
- The “ByteType “ type represent an 8-bit size.

typedef struct {
            ByteType       Location;              //Door Location number
            UInt16Type     Car_Number             //Indicate the Car Serial Number
} Door_Location_ID;
```

2.1.4.2.12 PTA_DMStoPTAErrorRsp

COMMAND Value: 50

DATA LENGTH: 2 bytes

The ***Slave*** end shall send this command response to the ***Master*** end whenever the previously received message request is invalid or the command is not recognized. The data section of this command response shall by formatted as defined in Table 2-27 and Table 2-28 below.

Table 2-27: PTA_DMStoPTAErrorRsp Data Bytes Definition

```
- The “ByteType” type represents an 8-bit size.

typedef struct _Err
{
            ByteType       Error_Code;
            ByteType       ucBadCmd;   //the requested command that is not supported
} Err;
```

Table 2-28: PTA_DMStoPTAErrorRsp Error Codes

Value	Description
0	Reserved
1	Reserved
2	Command not supported

2.1.4.2.13 PTA_TimeReq

COMMAND Value: 64

DATA LENGTH: 0 byte

The ***Master*** end shall send this command request to the ***Slave*** end to request its current time and date.

2.1.4.2.14 PTA_TimeRsp

COMMAND Value: 65

DATA LENGTH: 4 bytes

The ***Slave*** end shall send this command response to the ***Master*** end in response to the PTA_TimeReq command. The data section of this command is 4 bytes long. It is representing the time and date in standard “C” unsigned long type. This is a 32-bit number that represent the number of second elapsed since the January 1st, 1970 and is valid up to 2106.

2.1.4.2.15 PTA_SelfTestReq

COMMAND Value: 66

DATA LENGTH: 0 byte

The ***Master*** end shall send this command request to the ***Slave*** end to request self-test results.

2.1.4.2.16 PTA_SelfTestRsp

COMMAND Value: 67

DATA LENGTH: 2 bytes

The ***Slave*** end shall send this command response to the ***Master*** end in response to the PTA_SelfTestReq command request. The data section of this command is available in Table 2-29 below. Test_result1 to test_result8 and spares are project's specific tests and are defined in the project documentation.

Table 2-29: PTA_SelfTestRsp Data Bytes Definition

```
typedef struct {
    unsigned  char  test_result1                    :1;     // 1 bit
    unsigned  char  APP_CRC_test_result             :1;     // 1 bit
    unsigned  char  Boot_CRC_test_result            :1;     // 1 bit
    unsigned  char  SRAM_Pat_test_result            :1;     // 1 bit
    unsigned  char  SRAM_Add_test_result            :1;     // 1 bit
    unsigned  char  NVRAM_Pat_test_result           :1;     // 1 bit
    unsigned  char  NVRAM_Add_test_result           :1;     // 1 bit
    unsigned  char  test_result2                    :1;     // 1 bit, One Byte

    unsigned  char  test_result3                    :1;     // 1 bit
    unsigned  char  test_result4                    :1;     // 1 bit
    unsigned  char  test_result5                    :1;     // 1 bit
    unsigned  char  test_result6                    :1;     // 1 bit
    unsigned  char  test_result7                    :1;     // 1 bit
    unsigned  char  test_result8                    :1;     // 1 bit
    unsigned  char  Spare1_test_result              :1;     // 1 bit
    unsigned  char  Spare2_test_result              :1;     // 1 bit, One Byte
} Self_test;
```

2.1.4.2.17 PTA_CodeStatusReq

COMMAND Value: 74

DATA LENGTH: 1 byte

The Master end shall send this command request to the Slave end to request status (Size and CRC value) of all installed Software Configuration Items.

The data byte values are as per Table 2-30 below:

Table 2-30: PTA_CodeStatusReq Data Byte Definition

Value	Description
0	Reserved
1	Request Boot loader info
2	Request Application info
3 -> 255	Reserved for project' specific SCIs

2.1.4.2.18 PTA_CodeStatusRsp

COMMAND Value: 75

DATA LENGTH: 7 bytes

The ***Slave*** end shall send this command response to the ***Master*** end in response to the PTA_CodeStatusReq command request.

The data section of this command formatted as defined in Table 2-31 below.

Table 2-31: PTA_CodeStatusRsp Data Bytes Definition

```
typedef struct {

             unsigned char  ucSciReq;
             unsigned       long                    ulCodeSize;  //4 Bytes
             unsigned       int                     uiCrc;  // 2 Bytes

} Code_StatusStruct;
```

2.1.4.2.19 PTA_DMSConfigReq

COMMAND Value: 76

DATA LENGTH: 4 bytes

The ***Master*** end shall send this command request to the ***Slave*** end to sends or retrieves configuration information to/from the door module.

The data section is defined inTable 2-32, Table 2-33 and Table 2-34 below.

Table 2-32: PTA_DMSConfigReq Data Bytes Definition

```
typedef struct {
             unsigned       char                    ucCode;
             unsigned       char                    ucCommand;
             unsigned       int                     uiParameter;  // 2 Bytes
} DMS_ConfigStruct;
```

Table 2-33: PTA_DMSConfigReq *uCode* Field Values

uCode	Description
0	Reserved
1	Retrieve Door Control Module (DCM) configuration
2	Write Door Control Module (DCM) configuration
3	Retrieve Communication Gateway Module (CGM) configuration
4	Write Communication Gateway Module (CGM) configuration
5-255	Reserved for project's specific configuration information

Table 2-34: PTA_DMSConfigReq *uCommand* and *uiParameter* Fields Values

uCommand	Description	uiParamater
0	Reserved	-
1	Opening Time	0 to 255
2	Closing Time	0 to 255
3	Obstruction Retries	0 to 255
4	Obstruction Re-Opening Distance	0 to 255
5 -> 10	[Specific configuration parameter 1 to 6]	0 to 255
11 -> 255	Reserved for project's specific parameters	

2.1.4.2.20 PTA_DMSConfigRsp

COMMAND Value: 77

DATA LENGTH: 4 bytes

The Slave end shall send this command response to the Master end in response to the PTA_DMSConfigReq command request.

The data section of this command is formatted as in the command request defined in the previous section 2.1.4.2.19. The data from the command request is copied back to the command response. The programmed configuration is put in the response. If it changed, it is the same as the request, otherwise it might be different.

2.1.4.2.21 PTA_DoorCMDReq

COMMAND Value: 81

DATA LENGTH: 1 byte

The ***Master*** end shall send this command request to initiate door operations to the ***Slave*** end. This command request is only supported by the Door Control Module (DCM) and is ignored by other modules.

The data section definition is as per Table 2-35 below.

Table 2-35: PTA_DoorCMDReq Data Byte Definition

Value	Description
0	Reserved
1	Open Command
2	Close Command
3	Cycling command

2.1.4.2.22 PTA_DoorCMDRsp

COMMAND Value: 82

DATA LENGTH: 1 byte

The ***Slave*** end shall send this command response to the ***Master*** end to confirm the reception and execution of the requested command from the PTA_DoorCMDReq command request.

The data section is defined in the Table 2-36 below. The Door Control Module (DCM) only accepts these commands, all other modules shall respond with the value 255.

Table 2-36: PTA_DoorCMDRsp Data Byte Definition

Value	Description
0	Reserved
1	Open Command Accepted
2	Close Command Accepted
255	Command Rejected (no previous open command)

2.1.4.2.23 PTA_SystemCountersResetReq

COMMAND Value: 83

DATA LENGTH: 1 byte

The ***Master*** end shall send this command request to the ***Slave*** end to request the erasure of the System Counters.

The data section is formatted as defined in Table 2-37 below.

Table 2-37: PTA_SystemCountersResetReq Data Byte Definition

Value	Description
0	Reserved
1	Door Cycles Counter
2	Operating Hours
3	Event Occurrences
4-250	Reserved for project's specific counters
255	Reserved for failure

2.1.4.2.24 PTA_SystemCountersResetRsp

COMMAND Value: 84

DATA LENGTH: 1 byte

The ***Slave*** end shall send this command response in response to the ***Master*** end request for the reset of the System Counters.

The data section of this command response is identical to the data section of the command request as defined in Table 2-37.

APPENDIX A: CRC CALCULATION ALGORITHM

```
//  PURPOSE         : This function calculate the CRC16 of the DATA received.
//  INPUT(S)        : ucNewData = Is the new data to add on the old crc.
//                    uiOldCrc  = Is the old crc value to add.
//  RETURN          : The new crc calculated with the new data received.
unsigned int CRC_UpdtCrc (unsigned char ucNewData, unsigned int uiOldCrc)
{
    return (crctab[(uiOldCrc >> 8)^ucNewData] ^(uiOldCrc << 8));
}
```

```
const unsigned int near crctab[] = {
     0x0000,  0x1021,  0x2042,  0x3063,  0x4084,  0x50A5,  0x60C6,  0x70E7,
     0x8108,  0x9129,  0xA14A,  0xB16B,  0xC18C,  0xD1AD,  0xE1CE,  0xF1EF,
     0x1231,  0x0210,  0x3273,  0x2252,  0x52B5,  0x4294,  0x72F7,  0x62D6,
     0x9339,  0x8318,  0xB37B,  0xA35A,  0xD3BD,  0xC39C,  0xF3FF,  0xE3DE,
     0x2462,  0x3443,  0x0420,  0x1401,  0x64E6,  0x74C7,  0x44A4,  0x5485,
     0xA56A,  0xB54B,  0x8528,  0x9509,  0xE5EE,  0xF5CF,  0xC5AC,  0xD58D,
     0x3653,  0x2672,  0x1611,  0x0630,  0x76D7,  0x66F6,  0x5695,  0x46B4,
     0xB75B,  0xA77A,  0x9719,  0x8738,  0xF7DF,  0xE7FE,  0xD79D,  0xC7BC,
     0x48C4,  0x58E5,  0x6886,  0x78A7,  0x0840,  0x1861,  0x2802,  0x3823,
     0xC9CC,  0xD9ED,  0xE98E,  0xF9AF,  0x8948,  0x9969,  0xA90A,  0xB92B,
     0x5AF5,  0x4AD4,  0x7AB7,  0x6A96,  0x1A71,  0x0A50,  0x3A33,  0x2A12,
     0xDBFD,  0xCBDC,  0xFBBF,  0xEB9E,  0x9B79,  0x8B58,  0xBB3B,  0xAB1A,
     0x6CA6,  0x7C87,  0x4CE4,  0x5CC5,  0x2C22,  0x3C03,  0x0C60,  0x1C41,
     0xEDAE,  0xFD8F,  0xCDEC,  0xDDCD,  0xAD2A,  0xBD0B,  0x8D68,  0x9D49,
     0x7E97,  0x6EB6,  0x5ED5,  0x4EF4,  0x3E13,  0x2E32,  0x1E51,  0x0E70,
     0xFF9F,  0xEFBE,  0xDFDD,  0xCFFC,  0xBF1B,  0xAF3A,  0x9F59,  0x8F78,
     0x9188,  0x81A9,  0xB1CA,  0xA1EB,  0xD10C,  0xC12D,  0xF14E,  0xE16F,
     0x1080,  0x00A1,  0x30C2,  0x20E3,  0x5004,  0x4025,  0x7046,  0x6067,
     0x83B9,  0x9398,  0xA3FB,  0xB3DA,  0xC33D,  0xD31C,  0xE37F,  0xF35E,
     0x02B1,  0x1290,  0x22F3,  0x32D2,  0x4235,  0x5214,  0x6277,  0x7256,
     0xB5EA,  0xA5CB,  0x95A8,  0x8589,  0xF56E,  0xE54F,  0xD52C,  0xC50D,
     0x34E2,  0x24C3,  0x14A0,  0x0481,  0x7466,  0x6447,  0x5424,  0x4405,
     0xA7DB,  0xB7FA,  0x8799,  0x97B8,  0xE75F,  0xF77E,  0xC71D,  0xD73C,
     0x26D3,  0x36F2,  0x0691,  0x16B0,  0x6657,  0x7676,  0x4615,  0x5634,
     0xD94C,  0xC96D,  0xF90E,  0xE92F,  0x99C8,  0x89E9,  0xB98A,  0xA9AB,
     0x5844,  0x4865,  0x7806,  0x6827,  0x18C0,  0x08E1,  0x3882,  0x28A3,
     0xCB7D,  0xDB5C,  0xEB3F,  0xFB1E,  0x8BF9,  0x9BD8,  0xABBB,  0xBB9A,
     0x4A75,  0x5A54,  0x6A37,  0x7A16,  0x0AF1,  0x1AD0,  0x2AB3,  0x3A92,
     0xFD2E,  0xED0F,  0xDD6C,  0xCD4D,  0xBDAA,  0xAD8B,  0x9DE8,  0x8DC9,
     0x7C26,  0x6C07,  0x5C64,  0x4C45,  0x3CA2,  0x2C83,  0x1CE0,  0x0CC1,
     0xEF1F,  0xFF3E,  0xCF5D,  0xDF7C,  0xAF9B,  0xBFBA,  0x8FD9,  0x9FF8,
     0x6E17,  0x7E36,  0x4E55,  0x5E74,  0x2E93,  0x3EB2,  0x0ED1,  0x1EF0
};
```